



RAG Fundamentals For Data Engineers

What to learn • Why it matters • Where it fits



Shashank Mishra (Sr Data Engineer)



@_shashank_219



@shashank219

• visit www.growdataskills.com •

What should a Data Engineer learn in RAG?

Focus on the technical building blocks that make RAG reliable in real projects.

1



Knowledge ingestion

bring documents, tables and APIs into a searchable layer

2



Chunking & metadata

prepare content so retrieval can find the right context

3



Embeddings & indexes

convert meaning into vectors and store them efficiently

4



Retrieval & reranking

find and rank the most relevant evidence

5



Prompt + context

send clean, source-aware context to the LLM

6

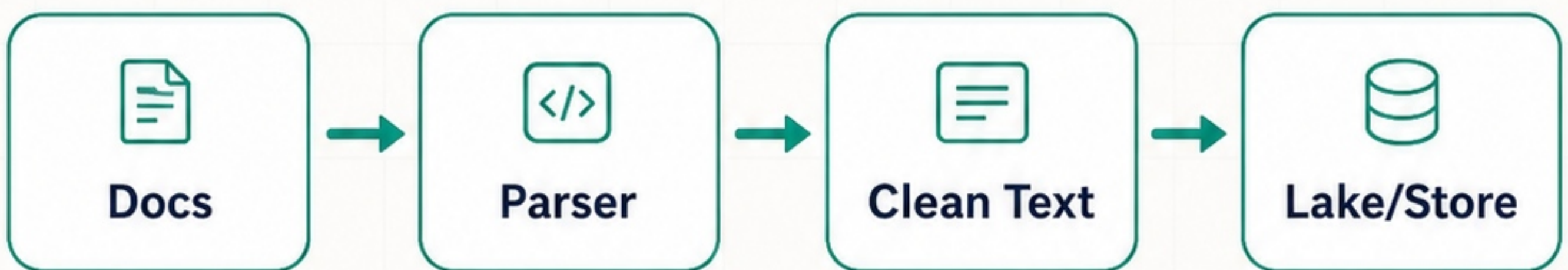


Evaluation & production

measure accuracy, latency, cost, security and freshness

1. Knowledge Ingestion

RAG starts before the LLM. You first need to collect and standardize enterprise knowledge.



What to learn

PDF/HTML/Markdown parsing, OCR needs, table extraction, API ingestion, batch vs incremental sync.



Used for

Keeping your RAG knowledge base updated from docs, wiki pages, tickets, databases and runbooks.



Data Engineering angle

Design this like a real ingestion pipeline: schema, checkpoints, retries, deduplication, data quality and failure alerts.

2. Chunking & Metadata

The same document can become useful or useless depending on how you split and tag it.



Chunking

Split content into meaningful blocks — section-wise, paragraph-wise, table-wise or semantic chunks.



Overlap

Keep small overlap between chunks so important context is not lost at boundaries.



Metadata

Attach source, owner, domain, date, product, region, access policy and version.



Used for

Better filtering, access control, freshness checks, source citation and debugging wrong answers.



Practical thumb rule

Start with 300–800 token chunks, then tune using retrieval evaluation — not guesswork.

3. Embeddings & Vector DB

Embeddings represent meaning numerically. Vector databases store and search those representations.

Text Chunk → Embedding Model → [0.12, -0.44, ...]



What to learn

Embedding models, vector dimensions, similarity search, cosine distance, ANN indexes like HNSW/IVF.



Vector DB role

Stores chunk embeddings + metadata and returns nearest chunks for a query.



Important

Changing embedding models usually means re-embedding and rebuilding your index. Plan versioning from day one.

4. Retrieval Strategy

Retrieval decides what evidence the LLM sees. This is where most RAG quality problems start.

1

Semantic search

Finds conceptually similar chunks even when exact words differ.

2

Keyword search

Great for IDs, error codes, names, SQL objects and exact terms.

3

Hybrid search

Combines vector + keyword search for stronger real-world retrieval.

4

Reranking

Uses a stronger model to reorder top results by relevance.



Used for

Improving factual grounding, reducing hallucination and finding the exact evidence needed to answer.

5. Context + Prompt Design

After retrieval, you must package the evidence cleanly before calling the LLM.

RAG prompt should include

-  **User question** — what the user wants
-  **Retrieved context** — ranked chunks + source
-  **Instruction** — answer only from context
-  **Output format** — summary, steps, SQL, JSON, table



What to learn

Prompt templates, context compression, citation formatting, no-answer behavior, guardrails.



Used for

Making answers grounded, consistent, auditable and safer for users.



More context $\neq$ better answer. Right context = better answer.

6. RAG Evaluation

You cannot improve RAG only by looking at random demos. You need measurable evaluation.

1

Retrieval Recall

Did we fetch the correct evidence?

2

Answer Faithfulness

Did the answer stay grounded in context?

3

Relevance

Did the answer solve the user's actual query?

4

Latency & Cost

How fast and expensive is each response?

DQ

What Data Engineers should build

Golden question-answer dataset, source-based test cases, retrieval logs, feedback loop, dashboards and alerting.

7. Production RAG Architecture

A reliable RAG system needs more than a notebook and a vector database.

1

Orchestration

Airflow, Step Functions, Databricks Workflows, cron-based sync



2

Security

RBAC, tenant isolation, PII masking, prompt injection checks



3

Freshness

incremental indexing, document versioning, deleted-doc handling



4

Observability

traces, retrieved chunks, token usage, latency, feedback



5

Caching

cache frequent retrievals and repeated LLM responses



6

Fallbacks

no-result answer, human review, ticket creation, escalation



RAG is not just an LLM topic.

It is a Data Engineering problem.

Learn ingestion, chunking, embeddings,
retrieval, reranking, prompt design,
evaluation and production monitoring.



SAVE
for revision



SHARE
with your team



FOLLOW
for more



@_shashank_219



@shashank219



visit www.growdataskills.com